



Welcome to ECE 4793/6793 and CSE 4353/6353: Applications of Literate Programming in Software Development. I'm excited to have you in this course, and I look forward to working with you throughout the semester. This is a hands-on, project-focused course designed to improve your programming skills by changing the way you think about writing software or firmware. Expect to write a lot of code, but also to explain your ideas so that others can contribute to your work. I welcome your comments, ideas, and participation in this course!

Instructor: Dr. Bryan A. Jones
Email: bjones@ece.msstate.edu
Office: Simrall 231

Office Hours: Tuesday, 3:30 – 4:30 PM or book an appointment via <https://bjones.youcanbook.me/>.

Lecture Time: Lecture MWF 10:00 AM Central Time, Simrall 106 (in-person/synchronous distance)

Distance Section: Both lecture and lab are synchronous (in-person and distance); see the course homepage for recordings.

Prerequisites: Grade of C or better in ECE 3724 or CSE 3724 and junior or graduate standing or consent of instructor

Corequisites: N/A

Textbook: None

Course Description and Objectives

(Prerequisites: Grade of C or better in ECE 3724 or CSE 3724 and junior or graduate standing or consent of instructor). Two hours lecture and two hours laboratory. Techniques for software development and embedded systems, with an emphasis on interleaved documentation for code longevity. (Same as CSE 4353/6353)

After successfully completing this course, the students will be able to:

- i. Discuss various approaches to literate programming; demonstrate knowledge of markup formats used in intermingling source code with structured comments; apply modern literate programming tools (3.1).
- ii. Demonstrate proficiency in literate programming methods, by developing projects in which documentation is interleaved with source code (3.3).
- iii. Utilize a distributed version control system to record the evolution of software using branches and commits; employ merge/rebase strategies to incorporate branches from other developers; define the differing locations in which a DVCS stores its data and the methods used to move data between these locations (3.6).
- iv. Employ social coding techniques to submit a pull request, write and respond to code reviews; apply software management techniques to coordinate decentralized development across multiple teams (3.5).

Methods of Evaluation and Standards of Achievement

Types of graded assignments include:

- (a) **Homework** – assigned and submitted through Canvas and GitHub.
- (b) **Project** – A series of projects, culminating in final project, will be assessed based on required functionality and on the documentation accompanying the project.



Graduate Student Requirements: Graduate students will be required to complete a project of greater complexity than the project required of undergraduate students: the project must involve significant algorithmic complexity, use of a high-complexity library, or changes to the software architecture of the project.

Distance student evaluation: Like students who attend in person, distance students will be evaluated based on submitted homework and projects. The uniqueness of each project assists in deterring academic misconduct.

Grading Breakdown	
Assignment Type	Percentage
Projects	40%
Homework	30%
Final project	30%

Grading Scale	
Grade	Average
A	90.0-100
B	80.0-89.9
C	70.0-79.9
D	60.0-69.9
F	<60.0

Note that final grades for group assignments (e.g., design project, presentations) can be adjusted by plus or minus 10% based on feedback collected from instructor assessment, in-class updates during sprints, a peer review or self-assessment process. Ideally, all team members contribute equally and, as a result, the team achieves their grade goals. Occasionally, peer review reveals that contributions are markedly uneven, despite all efforts to address the project as a team. As a part of the self-assessment process, team members will submit a written evaluation of their fellow team members. All claims of mutiny, insurgency, poor performance, etc., must be documented in sufficient detail to be given consideration.

LECTURE TOPICS (30 contact hours)

- I. Literate programming (10 hours)
 - A. Introduction to literate programming (LP)
 - B. Origins of LP
 - C. Current approaches to LP
 - D. Single-file approaches to LP
 - E. Project-based approaches LP
- II. Interleaved documentation and code longevity (10 hours)
 - A. Meta-code: believing is seeing
 - B. Writing to learn: documentation as design
 - C. Writing to teach: documentation as description
 - D. Writing to remember: documentation as database
 - E. Writing to reflect: documentation as introspection
- III. Tools and techniques for version control (10 hours)
 - A. Introduction to distributed version control systems (DVCS)
 - B. Storytelling through DVCS history: branches and commits
 - C. Collaborative techniques for DVCS: merge and rebase
 - D. Social coding: DVCS integration with GitHub
 - E. Software project management with GitHub



LABORATORY TOPICS (30 contact hours)

- I. Exploration of software development backgrounds (1 hour)
- II. Basics of LP (5 hours)
 - A. Coding with reflection
 - B. Introduction to LP
 - C. Introduction to CodeChat
 - D. Introduction to Sphinx
- III. Software development with DVCS (6 hours)
 - A. Introduction to DVCS
 - B. Introduction to GitHub
 - C. Social coding with LP
- IV. Project 1: development without LP requirements (6 hours)
 - A. Requirements and reflection on the existing code base
 - B. Scrum sessions
 - C. Demonstration and handoff
- V. Project 2: development with some LP requirements (6 hours)
 - A. Requirements and reflection on the existing code base
 - B. Scrum sessions
 - C. Demonstration and handoff (6 hours)
- VI. Final project: development with complete LP requirements
 - A. Requirements and reflection on the existing code base
 - B. Scrum sessions
 - C. Final demonstrations

CLASS INFORMATION

Laboratory Structure

The laboratory portion of this course consists of assignments to be submitted via Canvas and GitHub, accompanied by meetings with the instructor via MS Teams or an in-person appointment. Students must share their screens (if meeting via MS Teams) or bring a laptop (for in-person meetings) to discuss their code, review feedback on pull requests, and otherwise discuss the laboratory assignments.

Project Expectations

Projects consist of contributing either bug fixes or feature additions in the form of accepted pull requests to large, ongoing software development projects which utilize a wide variety of programming languages and libraries. Students are expected to quickly become conversant in new languages and libraries in order to provide meaningful contributions to these projects.

Students will work in groups on projects; groups may consist of students from differing campuses. Students should expect to meet with their teammates using MS Teams or other video conferencing software of the students' choice.

Course Materials

Course materials (recorded lectures, lecture slides, homework assignments, recorded problem sessions, and exams) will be accessed through Canvas and the course MS Teams site.



Expectations for the Classroom and Communication

The following policies for course communication apply for **ALL students**:

- You are required to check your MSU email account regularly. This is considered an official means of communication by MSU for distance education students.
- The course materials will be accessed through Canvas and MS Teams.
- Assignment submissions will utilize Canvas unless otherwise specified by the instructor.
- You are required to have access to a computer that connects to the internet.
- Students should direct correspondence directly related to the class to the instructor's MSU e-mail address.
- Students are encouraged to discuss homework together in a group, but the assignment should be completed individually unless otherwise specified.

The following policies for course communication apply to **students enrolled in the Distance section**:

- Faculty office hours will be hosted in MS Teams. Students can join using a computer or smartphone app.
- Students can correspond with each other via MS Teams. Please note that collaboration on individual projects is not acceptable.
- Students should expect to log in to Canvas and MS Teams no less than 3 times per week to access course information, lectures, and updates.

Minimum Technology Requirements

The following minimum technology requirements are necessary **for all students** to complete the course:

- Computer with web browser, Microsoft Office, and Adobe Reader
- Internet access

Distance students will also need:

- Webcam and microphone (computer or smartphone) to upload video responses to assignments or participate in virtual meetings / office hours.
- Video recording and editing software (Camtasia is available to download free from MSU ITS)

Technical Assistance

If you have questions about this course, please contact the instructor via Canvas messaging. For technical support (e.g., computer support, Canvas issues), please contact help@ece.msstate.edu or enr-dist-support@lists.msstate.edu or www.bagley.msstate.edu/distance.

Attendance Policy for Face-to-Face Instruction

Students registered in face-to-face sections are expected to attend all class meetings. Please refer Academic Operating policy 12.09 (<http://www.policies.msstate.edu/policypdfs/1209.pdf>) regarding attendance expectations and accommodations.

Attendance Policy for Distance Instruction

Distance students are expected to attend every class meeting by joining the live stream or watching assigned lecture videos and reading assigned material. Both lecture and lab meetings are synchronous. You must attend class and turn in assignments according to the weekly class schedule and assignment due dates.



MSU Syllabus via Canvas

The Mississippi State University Syllabus contains all policies and procedures that are applicable to every course on campus and online. The policies in the University Syllabus describe the official policies of the University and will take precedence over those found elsewhere. It is the student's responsibility to read and be familiar with every policy. The University Syllabus may be accessed at any time on the provost website under Faculty and Student Resources and at <https://www.provost.msstate.edu/faculty-student-resources/university-syllabus>.

Copyright

Copyrighted materials within the course are only for the use of students enrolled in the course for purposes associated with this course and may not be retained or further disseminated.

Course materials must not be posted on any website or added to any database without the instructor's written permission. Do not distribute test problems, homework, or any other materials. Do not post course materials on websites such as chegg.com, slader.com, etc. Violations of this policy will be referred to the Honor Court.

AI Policy: Permitted in This Course with Attribution

In this course, students are encouraged to use Generative AI Tools such as ChatGPT to support their work. Students must give credit and cite any AI-generated material according to rules of documentation including in-text citations, quotations, and references. All Github commits produced using AI (which should be the majority of your commits) should include a "Co-Authored-By:" line; for example, "Co-Authored-By: Claude Opus 5 <noreply@anthropic.com>".

Students must also include the following statement in assignments to indicate use of a Generative AI Tool: "The author acknowledges the use of [Generative AI Tool Name] in the preparation of this assignment for [brainstorming, grammatical correction, citation, etc.]."